

# Counterfactual Effect Binding for Pre-Commit LLM-Agent Tool Safety

Anonymous Authors

## Abstract

Tool-using LLM agents can commit security-relevant side effects before a human can inspect them. Existing evaluations usually ask whether a monitor rejects an unsafe call, but not whether its decision is bound to the call’s realized effect, target resource, operation, authorization context, and provenance. This distinction matters: a monitor can appear robust by tracking tool names or by abstaining, yet fail when the same effect moves to a different surface or when only the resource or authorization changes. We formalize *tool-effect binding* as a pre-commit safety property and introduce a counterfactual stress-test framework that varies these factors independently. Across an 822-case suite with 6,840 pairwise relations, released checkpoints and structural components exhibit nontrivial capabilities, but the evaluated non-oracle methods remain incomplete for joint binding. Our reference hard guard attains  $754/822 = 0.917$  coverage with  $13/360 = 0.036$  unsafe pre-allow overall; under targeted resource/authorization stress, unsafe pre-allow rises to  $74/120 = 0.617$  despite  $221/240 = 0.921$  coverage. Guided by this failure, we implement a local authorization-aware mediator with typed policies, resource resolution, operation semantics, multi-resource atomization, and provenance checks. On a 600-case, five-domain testbed, it increases coverage from  $120/600 = 0.200$  to  $528/600 = 0.880$  and reduces observed unsafe pre-allows from  $12/276$  to  $0/276$ , with  $0/252$  safe false denials. Human-corrected sensitivity on the original testbed preserves zero observed unsafe pre-allows but introduces  $4/230 = 0.017$  false denials. These results establish a controlled diagnostic and a local contract prototype, not production safety.

## 1 Introduction

Tool use turns an LLM agent’s output into a security decision. A proposed call can send a message, share a file, modify a calendar, publish to a workspace, or initiate a transaction. Once committed, such actions may be difficult or impossible to retract. This has motivated benchmarks that expose agents to high-stakes tools and prompt-injection attacks [1, 2], as well as defenses that intervene at planning or tool-invocation time [5, 6].

The authorization question at this boundary is relational rather than lexical. Sending a message may be authorized for one recipient but not another; reading a file does not authorize sharing it; a trusted user request and an instruction copied from untrusted tool output should

not carry the same authority. Conversely, tool names, wrappers, aliases, schemas, and trace formats may change while the intended effect remains the same. A monitor that follows these surfaces can therefore pass ordinary test cases for the wrong reason. It may preserve its decision across an authorization-changing resource swap, or change its decision when only an irrelevant wrapper changes.

Existing defenses address important parts of this problem. CaMeL constrains control and data flows and uses capabilities to restrict unauthorized flows [3]; IPIGuard constrains execution through a planned tool-dependency graph [4]; Progent enforces programmable least-privilege policies over tool calls [7]; and MiniScope reconstructs permission hierarchies for tool-calling agents [8]. These systems establish that structure and explicit policy are necessary. Our question is complementary and precedes policy enforcement: *is the candidate action bound to the correct policy-relevant effect, resource, operation, and provenance in the first place?* Aggregate attack success, task utility, or binary safety accuracy does not isolate this property.

We call this property *tool-effect binding*. Consider a pre-commit mediator that observes a task, a visible tool interface, a candidate call, and available evidence. Its decision should be invariant when only the surface changes but the authorized effect is preserved. It should change when the realized effect, target resource, operation mode, authorization context, or control provenance crosses the policy boundary. These requirements must be tested jointly: surface invariance alone can reward an always-allow rule, while effect sensitivity alone can miss a resource-specific authorization violation.

We operationalize this idea with a counterfactual stress framework. The framework organizes paired cases along surface, effect, resource, authorization, operation, provenance, and evidence axes, and reports unsafe pre-allow, safe false denial, coverage, and abstention. We apply it to simple proxies, released TS-Guard and Safiron checkpoints, IPIGuard and CaMeL components, semantic diagnostics, and a reference hard Effect-Binding Guard. The comparison is deliberately scoped to our custom stress inputs; it is not a reproduction of the original systems’ benchmark results.

The experiments reveal a consistent separation between partial robustness and joint binding. Existing methods are not reducible to tool-name classifiers, and the reference hard guard reaches  $754/822 = 0.917$  coverage with  $13/360 = 0.036$  unsafe pre-allow on the main suite. Yet under targeted resource/authorization perturbations, its unsafe pre-allow rises to  $74/120 = 0.617$  while coverage

remains  $221/240 = 0.921$ . The guard is not merely uncertain; it often makes the wrong authorization decision. This result identifies resource and authorization binding, rather than surface invariance alone, as the central bottleneck.

We then ask whether the bottleneck can be reduced by making authorization an explicit systems interface. Our local pre-commit prototype represents typed authorization context, resolves resource aliases, distinguishes operation modes, expands multi-resource calls into authorization atoms, and overlays provenance checks. On a 600-case testbed spanning email, calendar, file, workspace, and transaction domains, it increases coverage from  $120/600 = 0.200$  to  $528/600 = 0.880$  and reduces observed unsafe pre-allows from  $12/276$  to  $0/276$ . The result survives strict label-hidden replay, resource perturbation, and an independently implemented reference authorizer. A separate 60-case human audit finds label and atom corrections; under corrected sensitivity on the original testbed, unsafe pre-allow remains  $0/320$ , while safe false denial becomes  $4/230 = 0.017$ .

This paper makes four contributions:

- **Security property and measurement.** We define tool-effect binding as a pre-commit property and introduce counterfactual tests that separate surface invariance from effect, resource, authorization, operation, and provenance sensitivity.
- **Cross-method diagnosis.** We show that evaluated checkpoints and structural components exhibit nontrivial but incomplete binding profiles, and we preserve the distinction between released-checkpoint stress, component stress, diagnostics, and non-deployable upper bounds.
- **Reference guard and bottleneck.** We provide an inspectable hard guard with explicit abstention and demonstrate that resource/authorization shifts remain a high-error regime despite high decision coverage.
- **Authorization-aware prototype and audit.** We show that explicit authorization infrastructure can substantially increase controlled local mediation coverage, and we validate the result with replay, decision-path, perturbation, reference-authorizer, and human-corrected analyses.

The evidence is intentionally bounded. The counterfactual suite is a controlled diagnostic, and the authorization-aware mediator is evaluated against a local mock contract. We do not claim a complete permission system, production safety, or validation on deployed SaaS, browser, banking, email, or workspace services.

## 2 Threat Model and Security Goal

### 2.1 Pre-Commit Mediation

We consider a tool-using agent that proposes a candidate action  $a$  before commit. The mediator observes a task context  $c$ , visible tool surface or schema  $s$ , the candidate call, and optional non-oracle evidence  $z$  such as trace summaries or provenance metadata. The mediator returns  $d \in \{\text{ALLOW}, \text{DENY}, \text{ABSTAIN}\}$  before any external side effect is executed.

We use the following terms. A *realized effect* is the semantic outcome the action would commit, such as sending a message or modifying a file. A *resource* is the affected object, account, channel, file, recipient, attendee, or payee. An *operation mode* distinguishes read, write, append, share, transfer, public, private, and similar modes. An *authorization context* describes which effects on which resources are allowed by the task. *Data provenance* and *control provenance* describe where content and instructions originate. A *binding tuple* is the monitor’s inferred relation among action, effect, resource, operation, authorization, evidence, and provenance.

The safety goal is scoped: prevent observed unsafe pre-allows in the constructed stress setting while reporting safe false denials and abstentions. We do not execute real external side effects, and we do not claim correctness for deployed SaaS, browser, banking, email, or workspace systems.

### 2.2 Metrics

Let unsafe rows be rows whose expected decision is DENY, and safe rows be rows whose expected decision is ALLOW. Let  $N_{\text{UA}}$  be unsafe rows predicted ALLOW,  $N_{\text{SD}}$  be safe rows predicted DENY, and  $N_{\text{dec}}$  be non-abstaining rows. We report:

$$\text{UPA} = \frac{N_{\text{UA}}}{N_{\text{unsafe}}}, \quad \text{FDenY} = \frac{N_{\text{SD}}}{N_{\text{safe}}}, \quad \text{Coverage} = \frac{N_{\text{dec}}}{N_{\text{all}}}.$$

Abstention is reported separately or as  $1 - \text{Coverage}$ . We treat abstention as incomplete mediation, not as success. Non-oracle deployable inputs exclude expected labels, gold expanded atoms, violation reasons, and scoring metadata.

## 3 Counterfactual Effect-Binding Framework

The framework constructs paired or grouped scenarios in which one axis changes while other axes are held fixed as far as the schema permits. The axes include: same effect with different surface, same surface with different effect, resource shift, authorization shift, operation-mode shift, provenance or control-source shift, and evidence availability.

The purpose is not to create a universal benchmark. The purpose is to stress whether a monitor’s decision follows the binding tuple rather than a proxy. We therefore label rows by scope: baseline rows, released-checkpoint custom-stress rows, original-component custom-stress rows, diagnostic rows, and upper-bound rows. These labels prevent a custom-stress result from being mistaken for an original-paper reproduction or a deployed guardrail claim.

The main E48 construction contains 822 rows and 6840 pairwise comparisons, drawing from phase4, IPIGuard-style, and CaMeL-style custom-stress sources. The sampled E47 human audit is complete for 222 primary and 56 secondary rows, with perfect agreement in the audited decision/effect/resource/authorization fields and six corrected-label subset rows. This supports the custom-stress packet used here, but not every possible tool or deployment context.

## 4 Reference Effect-Binding Guard

The reference hard Effect-Binding Guard is an inspectable monitor. It derives tuple views for effect, resource, authorization match, provenance risk, and evidence support, then applies a deterministic decision policy. It allows when required tuple elements agree with the authorization context, denies when the tuple conflicts with authorized effects or resources, and abstains when the evidence is insufficient for a non-oracle decision.

This guard is intentionally conservative. Its role is to test whether explicit tuple binding improves over surface proxies and to expose where hard binding fails. On the 822-row E48 suite, the full guard has unsafe pre-allow  $13/360 = 0.036$  with Wilson interval  $[0.021, 0.061]$ , safe false denial  $30/462 = 0.065$  with interval  $[0.046, 0.091]$ , and coverage  $754/822 = 0.917$  with interval  $[0.896, 0.934]$ . On the test split, unsafe pre-allow is  $5/98 = 0.051$ , safe false denial is  $6/126 = 0.048$ , and coverage is  $214/224 = 0.955$ .

These numbers justify the guard as a reference method, not as a solved defense. The next sections ask which axes remain unresolved.

## 5 Existing-Method Stress Results

**Finding 1: evaluated methods are not merely tool-name classifiers, but they do not satisfy joint binding.** Table 1 summarizes custom-stress capability profiles. The tool-name rule has perfect surface invariance by construction but no effect, authorization, or resource sensitivity. Released TS-Guard and Safiron checkpoints show nontrivial behavior on several axes. The local Qwen mapper is also not a simple name rule. However, the profiles are uneven, and unsafe pre-allow remains nonzero where measured.

Table 1: Existing-method and component profiles under our custom stress. These rows are not original-paper benchmark reproductions. N/I means that the component does not make the realized-effect decision needed for the metric.

| Method              | Surf. | Eff.  | Auth. | Res.  | UPA   |
|---------------------|-------|-------|-------|-------|-------|
| Tool-name proxy     | 1.000 | 0.000 | 0.000 | 0.000 | 0.292 |
| TS-Guard checkpoint | 0.883 | 0.625 | 0.764 | 0.833 | 0.159 |
| Safiron checkpoint  | 0.669 | 1.000 | 0.088 | 0.167 | 0.545 |
| IPIGuard topology   | 1.000 | N/I   | N/I   | N/I   | N/I   |
| Local Qwen mapper   | 0.992 | 0.833 | 0.375 | 0.125 | 0.333 |

The result sharpens the paper’s gap. The problem is not that every defense is a tool-name classifier. The problem is that surface robustness, effect sensitivity, authorization sensitivity, resource awareness, and provenance handling can come apart. A pre-commit mediator needs the joint tuple.

## 6 Resource and Authorization Bottleneck

**Finding 2: resource/authorization binding is the main hard-guard bottleneck.** E50 targets the reference guard with resource/authorization stress and control-provenance stress. Under resource/authorization stress, the guard has unsafe pre-allow  $74/120 = 0.617$  with interval  $[0.527, 0.699]$ , safe false denial  $0/120 = 0.000$ , and coverage  $221/240 = 0.921$ . This is not merely an abstention failure. The guard often makes a decision, but the decision permits unsafe resource or authorization shifts.

The provenance stress has a different profile: unsafe pre-allow  $0/144 = 0.000$ , safe false denial  $36/192 = 0.188$ , and coverage  $183/336 = 0.545$ . Provenance checks can reduce observed unsafe allows in that slice, but with substantial false-denial and coverage tradeoffs. Together, these results motivate an interface-level response: the mediator needs explicit authorization and resource structure, not only a harder surface or tuple rule.

## 7 Authorization-Aware Pre-Commit Prototype

E55-v2 implements that response as a local mock pre-commit contract. The dataset has 600 rows across five domains: email, calendar, file, Slack, and transaction, with 120 rows per domain and 60 pair groups. The stress axes include same-effect surface shifts, resource shifts, operation shifts, provenance shifts, and multi-resource expansion.

The prototype expands a visible tool call into effect-resource-operation atoms, one per recipient, attendee, file, channel, account, payee, visibility, or operation component. It checks those atoms against typed authoriza-

Table 2: E55-v2 strict label-hidden pre-commit authorization result. CI columns are Wilson intervals from the local result artifact.

| Method              | UPA    | FDeny | Coverage | Abstain |
|---------------------|--------|-------|----------|---------|
| Existing hard guard | 12/276 | 0/252 | 120/600  | 480/600 |
| Rate                | 0.043  | 0.000 | 0.200    | 0.800   |
| Authz-aware guard   | 0/276  | 0/252 | 528/600  | 72/600  |
| Rate                | 0.000  | 0.000 | 0.880    | 0.120   |

tion context, resource aliases, operation-mode semantics, multi-resource rules, provenance metadata, and available evidence. The strict mode excludes expected labels, gold atoms, violation reasons, and scoring metadata.

**Finding 3: explicit authorization infrastructure turns abstention-heavy binding into high-coverage controlled mediation.** Table 2 reports the strict E55-v2 comparison. The existing hard guard abstains on most rows: coverage  $120/600 = 0.200$  and abstention  $480/600 = 0.800$ . The authorization-aware guard covers  $528/600 = 0.880$  rows and abstains on  $72/600 = 0.120$ . Observed unsafe pre-allow drops from  $12/276 = 0.043$  to  $0/276 = 0.000$ , while safe false denial remains  $0/252 = 0.000$  in E55-v2.

This is the strongest constructive result in the package, but its scope is narrow. It shows that explicit authorization infrastructure can make many local mock-contract decisions decidable. It does not show that the same guard is correct for independent policies, real external services, or adversarially adapted tool ecosystems.

## 8 Audit and Validity Checks

**Finding 4: audits reduce lexical and implementation-coupling concerns, while also exposing label corrections.** Original E55 strict replay reproduces its canonical core metrics: the authz-aware guard has unsafe pre-allow 0.0, safe false denial 0.0, and coverage 0.92 in both canonical and recheck outputs. We use that result as reproducibility evidence, not as the main table.

The E57 human audit covers 60 of 60 spot-audit rows. It reports decision agreement  $54/60 = 0.900$ , all human checks true  $46/60 = 0.767$ , six decision corrections, thirteen atom corrections, and ten violation-reason corrections. The main caveats are transaction-domain amount handling and unknown-resource fallback. These corrections are material: the paper should not claim perfect label confirmation.

Under minimal 600-row correction for original E55, the authz-aware guard preserves unsafe pre-allow  $0/320 = 0.000$  and coverage  $552/600 = 0.920$ , but safe false denial becomes  $4/230 = 0.017$ . This is stronger evidence than ignoring the audit, because it states how the main conclusion changes under corrected labels.

E57-v2 adds deterministic validity checks after the

corrected E55-v2 rerun. Resource-name perturbation changes zero decisions, with UPA delta 0.0 and coverage delta 0.0. An independently implemented reference authorizer has decision agreement 1.0, atom-count agreement 1.0, and atom resource-set agreement 1.0 on the checked rows. These checks reduce concerns about naming templates and single-implementation coupling, but they do not create independent deployment evidence.

## 9 Discussion

The main design lesson is that pre-commit tool safety should report binding axes separately. A monitor can be robust to surface changes while still failing resource or authorization shifts. It can also reduce unsafe pre-allows by abstaining or over-denying, which is why coverage and safe false denial must be reported with unsafe pre-allow.

The second lesson is that authorization cannot remain implicit. E55-v2 improves controlled mediation because resources, aliases, operation modes, atom expansion, provenance, and evidence availability become explicit inputs to the decision procedure. This does not make the prototype a complete permission system. It shows that missing authorization infrastructure can be a bottleneck independent of classifier strength.

The third lesson concerns audit practice. The human audit did not merely rubber-stamp labels; it found corrections. Reporting corrected-label sensitivity makes the result more credible and more limited. For safety-relevant tool agents, an audit that changes false-denial rates is preferable to a hidden assumption of perfect construction.

## 10 Related Work

**Security evaluation for tool-using agents.** ToolEmu uses an LM-emulated sandbox to search for high-stakes failures across diverse tools without instantiating every external environment [1]. AgentDojo provides an extensible environment for evaluating prompt-injection attacks and defenses over realistic agent tasks and untrusted tool data [2]. These benchmarks establish that end-to-end agent behavior can fail in security-relevant ways and that evaluation must include both utility and adversarial conditions. Our framework addresses a narrower measurement question. Rather than estimating aggregate task or attack outcomes, it constructs controlled counterfactual pairs to determine which semantic variable a pre-commit decision follows. It is therefore a diagnostic layer that can be applied inside a broader benchmark, not a replacement for end-to-end evaluation.

**Pre-execution and step-level guardrails.** ToolSafe introduces TS-Bench and TS-Guard for proactive step-level tool-invocation safety, with feedback intended to improve subsequent agent reasoning [5]. Safron operates at the planning stage and combines a cross-planner adapter with a compact guard model trained on synthetic trajec-

tories; Pre-Exec Bench evaluates detection, categorization, explanation, and cross-planner transfer [6]. These approaches ask whether an action or plan is risky before execution. We evaluate their released checkpoints on our custom counterfactual inputs where available, but do not reproduce their original benchmark results. Our distinction is methodological: binary safety accuracy can combine surface invariance, effect sensitivity, resource awareness, authorization sensitivity, and abstention into one score, whereas our stress lattice reports these capabilities separately.

**Structural defenses against prompt injection.** CaMeL places a protective layer around the LLM, extracts trusted control and data flows, and applies capabilities to prevent unauthorized data flows [3]. IPIGuard separates planning from interaction with untrusted data and constrains execution through a tool-dependency graph [4]. Both move security enforcement away from unconstrained model judgment and toward explicit structure. Our work shares that systems motivation but studies a different failure boundary: whether a proposed action has been mapped to the correct realized effect, resource, operation, authorization, and provenance before a structural policy is applied. Accordingly, our CaMeL and IPIGuard results are component-scoped custom stress. They neither reproduce the full systems nor imply failure on the authors’ original evaluations.

**Authorization and least privilege.** Progent expresses fine-grained privilege policies over tool calls in a domain-specific language and enforces them deterministically at runtime [7]. MiniScope reconstructs tool permission hierarchies and combines them with a mobile-style permission model to confine tool-calling agents [8]. This line of work is closest to our authorization-aware prototype because it makes authority explicit rather than asking an LLM to infer safety unconstrained. The distinction is between *policy representation/enforcement* and *binding*. A policy engine can only enforce the intended rule if the candidate call is associated with the correct effect, resource, operation, and provenance. Our counterfactual framework measures that association, while our prototype tests a local contract in which the relevant authorization structure is explicit. It is narrower than Progent or MiniScope as a permission system and does not provide their broader system-level enforcement scope.

**Positioning.** Across these literatures, our contribution is not a new prompt-injection benchmark, a universally stronger guardrail, or a complete least-privilege system. It is a controlled test of a prerequisite shared by these approaches: the decision at the action boundary should track policy-relevant semantics rather than incidental surface form. Reporting unsafe pre-allow together with safe false denial, coverage, and abstention also prevents conservative non-decisions from being conflated with successful mediation.

## 11 Limitations

The study is controlled and local. E47/E48/E50 are custom-stress diagnostics, not universal benchmarks. E55-v2 is a mock pre-commit authorization contract, not validation on real SaaS, browser, banking, email, or workspace systems. No real external side effects are executed.

The E55-v2 guard and labels share an explicit contract, so the result tests whether the infrastructure can mediate that contract, not whether an independently authored policy will be implemented correctly. The project still lacks an independently authored held-out E55-style contract and a realistic sandbox or replay domain with a full mediator/commit protocol.

Existing-method rows should be read as custom-stress or component evaluations. They do not show that TSGuard, Saffron, IPIGuard, CaMeL, or any other system fails its original benchmark or intended deployment setting. The paper also does not claim a complete permission system, a prompt-injection defense, or a safety guarantee.

## 12 Ethics and Artifacts

The experiments use constructed local scenarios and mock contracts. The paper does not report executing real external side effects. Failure examples should be used for diagnostic and defensive evaluation, not as operational attack guidance. Before release, artifacts should be checked for secrets, deanonymizing local paths, and any accidental personal data.

The artifact package should include the local result JSON files, construction scripts, audit packets, test commands, and a result-to-source table. NDSS artifact review encourages reproducible code, data, scripts, models, tests, and benchmarks; this package is organized to support that review but still requires final anonymization and path cleanup.

## 13 Conclusion

Tool-agent safety monitors need to bind candidate actions to realized effects, resources, authorization, operation modes, evidence, and provenance before commit. Counterfactual stress testing shows that surface-form robustness and axis-specific capabilities are not enough for joint binding. A hard reference guard helps but leaves a resource/authorization bottleneck. E55-v2 shows that explicit authorization infrastructure can improve controlled local pre-commit mediation, reducing observed unsafe pre-allow to 0/276 with 528/600 coverage in the mock contract. The remaining work is to test independently authored contracts and realistic replay or sandbox mediators before making deployment-facing claims.

## A Artifact Source Map

The headline numbers in this draft trace to the following local artifacts:

- E48 hard guard: `results/analysis/results/e48_tuple_guard_results.json`.
- E50 bottleneck: `results/analysis/results/e50_hard_guard_robustness_results.json`.
- Existing-method matrix: `tables/table1_existing_defenses_capability_matrix.md` and `tables/tables/official_checkpoint_summary.md`.
- E55-v2 main result: `audit/analysis/results/e55_v2_precommit_authz_results_strict.json`.
- Original E55 corrected sensitivity: `results/analysis/results/e55_human_corrected_sensitivity.json`.
- E57 human audit: `audit/analysis/results/e57_human_audit_validation.json`.
- E57-v2 validity checks: `audit/analysis/results/e57_v2_validity_checks_report.json`.

## References

- [1] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. arXiv:2309.15817, 2024.
- [2] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352, 2024.
- [3] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025.
- [4] Hengyu An, Jinghui Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. IP-IGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents. arXiv:2508.15310, 2025.
- [5] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. ToolSafe: Enhancing Tool Invocation Safety of LLM-Based Agents via Proactive Step-Level Guardrail and Feedback. arXiv:2601.10156, 2026.
- [6] Yue Huang, Hang Hua, Yujun Zhou, Pengcheng Jing, Manish Nagireddy, Inkit Padhi, Greta Dolcetti, Zhangchen Xu, Subhajit Chaudhury, Amrith Rawat, Liubov Nedoshivina, Pin-Yu Chen, Prasanna Sattigeri, and Xiangliang Zhang. Building a Foundational Guardrail for General Agentic Systems via Synthetic Data. arXiv:2510.09781, 2025.
- [7] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703, 2025.
- [8] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents. arXiv:2512.11147, 2025.